

Laboratorio 6

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [ ]: # %pip install -r requirements.txt
# jupyter nbconvert lab6.ipynb --to html
```

Task 1

Como Ingeniero Principal (Lead AI Engineer) del proyecto AgriTech, usted debe justificar las decisiones arquitectónicas ante su equipo y sus clientes. Responda a los siguientes escenarios en su reporte (máximo 1 página por respuesta), combinando la teoría matemática con el pragmatismo laboral.

Inciso 1

Un desarrollador junior de su equipo sugiere:

"Para detectar con mayor precisión las texturas de las hojas enfermas, deberíamos construir una red secuencial clásica (tipo VGG) pero de 150 capas. Más profundo siempre es mejor".

Como líder técnico, explíquelo argumentativamente por qué esta red fracasará estrepitosamente en el entrenamiento (mencionando el fenómeno de degradación y el desvanecimiento del gradiente). Luego, justifique cómo la adición estructural de las conexiones residuales ($F(x) + x$) de ResNet rescata el proyecto, haciendo viable entrenar redes ultra-profundas sin colapsar.

La propuesta de una VGG secuencial de 150 capas parte de una idea incompleta porque más profundidad no garantiza mejor aprendizaje. En redes planas muy profundas aparece el desvanecimiento del gradiente el cual al multiplicarse muchas derivadas en backpropagation, el gradiente tiende a cero y las primeras capas dejan de aprender.

Más grave aún es la degradación. ResNet mostró que al aumentar la profundidad en arquitecturas planas no solo puede empeorar la generalización, sino incluso el error de entrenamiento. Esto no es overfitting, sino un problema de optimización: aunque una red profunda podría imitar a una más pequeña dejando capas como identidad, el optimizador no logra encontrar esa solución. La red se vuelve más difícil de entrenar, no más efectiva.

ResNet soluciona esto reformulando el problema. En lugar de aprender $H(x)$ directamente, cada bloque aprende el residuo:

$$F(x) = H(x) - x$$

y la salida se define como:

$$y = F(x) + x$$

Si la transformación adicional no aporta valor, basta con que $F(x) \rightarrow 0$ y el bloque implementa identidad fácilmente.

En retropropagación aparece la clave:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial H} \left(\frac{\partial F}{\partial x} + 1 \right)$$

Ese $+1$ crea un camino directo para el gradiente, reduciendo el desvanecimiento y evitando la degradación.

Por lo que una VGG de 150 capas fracasaría por problemas de optimización. Las conexiones residuales hacen viable entrenar redes ultra-profundas porque estabilizan el flujo del gradiente y convierten la profundidad en una ventaja real, no en un obstáculo.

Inciso 2

Las enfermedades en las hojas de mango son visualmente heterogéneas: La Antracnosis se presenta como puntos negros diminutos, mientras que el Moho Polvoriento cubre áreas enormes de la hoja. Analice cómo la topología en paralelo del módulo Inception (usando filtros 3×3 y 5×5 simultáneamente) es ideal para este problema biológico en particular. Además, desde una perspectiva de costos de infraestructura (uso de GPUs en AWS o Google Cloud), explique cómo la inserción estratégica de convoluciones de 1×1 evita la explosión de la dimensionalidad y salva el presupuesto mensual de la startup.

Las enfermedades de la hoja de mango no aparecen en una sola escala. La **Antracnosis** genera puntos pequeños y localizados, mientras que el **Moho Polvoriento** cubre zonas amplias. Por eso, usar un único tamaño de filtro sería una decisión rígida para un problema que es claramente **multi-escala**.

El módulo **Inception** resuelve esto con su estructura en paralelo. En lugar de aplicar un solo filtro, procesa la misma imagen al mismo tiempo con filtros 3×3 y 5×5 (entre otros) y luego concatena los

resultados. Así, la red puede detectar detalles pequeños y, a la vez, patrones más grandes dentro del mismo bloque.

- Los filtros 3×3 son adecuados para capturar puntos pequeños, bordes cortos y cambios locales de textura, como los de la Antracnosis.
- Los filtros 5×5 integran una región más amplia y detectan manchas extendidas o coberturas grandes, como el Moho Polvoriento.

La arquitectura no obliga a elegir entre detalle fino o contexto amplio: aprende ambos en paralelo.

Ahora bien, aplicar directamente filtros 3×3 y 5×5 sobre muchos canales sería muy costoso. El costo aproximado de una convolución es:

$$k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

Al pasar de 3×3 a 5×5 , el factor espacial sube de 9 a 25. Si además hay muchos canales, el número de operaciones crece rápido y el uso de memoria también.

Aquí entran las convoluciones 1×1 , que actúan como reductores de canales antes de aplicar los filtros grandes. Primero comprimen la información y luego se ejecutan las ramas costosas. Si reducimos de C_{in} a C_r canales (con $C_r \ll C_{in}$), el costo baja mucho:

Sin reducción:

$$25 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

Con reducción previa:

$$1 \cdot C_{in} \cdot C_r \cdot H \cdot W + 25 \cdot C_r \cdot C_{out} \cdot H \cdot W$$

La diferencia es grande cuando C_r es mucho menor que C_{in} .

Desde el punto de vista de infraestructura, menos operaciones significan menos tiempo por época, menor uso de memoria y menor necesidad de GPUs grandes. En proveedores cloud eso se traduce directamente en menos costo mensual. Sin las 1×1 , el modelo podría volverse demasiado caro para una startup.

Inciso 3

El cliente final (el agricultor) usará la aplicación en un teléfono Android de gama baja en medio del campo, sin conexión a internet. Sabemos que MobileNet logra esta eficiencia gracias a la Depthwise Separable Convolution. Describa brevemente cómo esta convolución divide el trabajo (filtrado espacial vs. combinación de canales). Sin embargo, en ingeniería no hay soluciones mágicas, todo tiene trade-offs.

El costo aproximado de una convolución tradicional es:

$$k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

donde el filtro de tamaño $k \times k$ opera sobre todos los canales de entrada para producir todos los canales de salida. Cuando C_{in} y C_{out} crecen, el número de operaciones aumenta rápidamente, lo que impacta tiempo de inferencia y consumo de memoria.

MobileNet reduce este costo usando **Depthwise Separable Convolution**, que divide el trabajo en dos etapas.

Primero aplica la **Depthwise Convolution**, que realiza solo el **filtrado espacial**. Se usa un filtro por cada canal de entrada, sin mezclar información entre canales. Su costo es:

$$k^2 \cdot C_{in} \cdot H \cdot W$$

Aquí se detectan bordes, texturas o patrones locales dentro de cada canal, pero no hay combinación entre ellos.

Luego aplica la **Pointwise Convolution** (1×1), que se encarga de la **combinación de canales**. Esta etapa aprende cómo integrar las características obtenidas en cada canal. Su costo es:

$$C_{in} \cdot C_{out} \cdot H \cdot W$$

El costo total queda:

$$k^2 \cdot C_{in} \cdot H \cdot W + C_{in} \cdot C_{out} \cdot H \cdot W$$

Comparado con la convolución estándar, la reducción es grande, especialmente cuando $k = 3$. En la práctica, el número de operaciones puede disminuir entre 8 y 9 veces. Eso se traduce en menor latencia, menor consumo de batería y modelos más pequeños, algo clave para ejecutar inferencia directamente en el teléfono del agricultor.

Task 2

Utilice PyTorch o TensorFlow/Keras (a su elección). Debe escribir el código desde cero o basarse en las documentaciones oficiales. Ejecute sus experimentos en Google Colab, Kaggle Notebooks o en su GPU local. Siéntanse libres de hacer uso de IA de forma educada, es decir, entendiendo realmente que lo que esté haciendo sea realmente lo que necesitan y que sobretodo lo entiendan.

1. Descargue el dataset, divídalo en Entrenamiento (70%), Validación (15%) y Prueba (15%).

Implemente Data Augmentation (rotaciones, flips, recortes) vital para evitar el sobreajuste en imágenes agrícolas.

2. Cargue los siguientes modelos pre-entrenados en ImageNet y congele sus capas base, reemplazando solo el cabezal de clasificación para nuestras 8 clases de mango:

- a. ResNet (Puede usar ResNet50).
- b. Inception (Puede usar InceptionV3).
- c. MobileNet (Puede usar MobileNetV2 o V3-Small/Large).

3. Entrene los 3 modelos utilizando la misma función de pérdida (Cross-Entropy) y optimizador (ej. Adam) por un máximo de 15 a 20 épocas (o use Early Stopping).
4. Registre las métricas correspondientes para cada modelo:
 - a. Accuracy (Exactitud) en el conjunto de prueba.
 - b. F1-Score (Macro) en el conjunto de prueba.
 - c. Tamaño del modelo final (en Megabytes) al guardarlo en disco (.pth o .h5).
 - d. Tiempo de Inferencia: Cuántos milisegundos (ms) tarda en predecir una sola imagen (promedio sobre 100 imágenes).

```
In [6]: import os
import time
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms, models
from sklearn.metrics import classification_report, f1_score, accuracy_score
import warnings
warnings.filterwarnings("ignore")

# 1. Preparación de Datos
dataset_dir = 'archive' # Ruta a La carpeta del dataset

def get_dataloaders(img_size=224, batch_size=32):
    # Transformaciones con Data Augmentation para entrenamiento
    train_transforms = transforms.Compose([
        transforms.RandomResizedCrop(img_size),
        transforms.RandomHorizontalFlip(),
        transforms.RandomVerticalFlip(),
        transforms.RandomRotation(20),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    # Transformaciones para validación/prueba (sin aumento)
    test_transforms = transforms.Compose([
        transforms.Resize(img_size + 32),
        transforms.CenterCrop(img_size),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    # Cargar todo el dataset temporalmente para dividirlo
    full_dataset = datasets.ImageFolder(dataset_dir)
    num_classes = len(full_dataset.classes)

    # División: 70% Entreno, 15% Validación, 15% Prueba
    total_size = len(full_dataset)
    train_size = int(0.7 * total_size)
    val_size = int(0.15 * total_size)
    test_size = total_size - train_size - val_size
```

```

# Generador manual para reproducibilidad
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset, test_dataset = random_split(
    full_dataset, [train_size, val_size, test_size], generator=generator
)

# Clase auxiliar para aplicar transformaciones correctas a cada split
class DatasetWrapper(torch.utils.data.Dataset):
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform
    def __getitem__(self, index):
        x, y = self.subset[index]
        if self.transform:
            x = self.transform(x)
        return x, y
    def __len__(self):
        return len(self.subset)

train_data = DatasetWrapper(train_dataset, transform=train_transforms)
val_data = DatasetWrapper(val_dataset, transform=test_transforms)
test_data = DatasetWrapper(test_dataset, transform=test_transforms)

train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True, num_workers=0)
val_loader = DataLoader(val_data, batch_size=batch_size, shuffle=False, num_workers=0)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False, num_workers=0)

return train_loader, val_loader, test_loader, num_classes

```

2. Carga de Modelos

```

def get_model(model_name, num_classes):
    if model_name == 'resnet50':
        weights = models.ResNet50_Weights.IMAGENET1K_V1
        model = models.resnet50(weights=weights)
        # Congelar capas base
        for param in model.parameters():
            param.requires_grad = False
        # Reemplazar cabezal
        in_features = model.fc.in_features
        model.fc = nn.Linear(in_features, num_classes)
        img_size = 224

    elif model_name == 'inception_v3':
        weights = models.Inception_V3_Weights.IMAGENET1K_V1
        model = models.inception_v3(weights=weights, aux_logits=True)
        # Congelar capas base
        for param in model.parameters():
            param.requires_grad = False
        # Reemplazar cabezales principal y auxiliar
        model.AuxLogits.fc = nn.Linear(model.AuxLogits.fc.in_features, num_classes)
        model.fc = nn.Linear(model.fc.in_features, num_classes)
        img_size = 299 # Inception requiere 299x299

    elif model_name == 'mobilenet_v3':
        weights = models.MobileNet_V3_Large_Weights.IMAGENET1K_V1

```

```

model = models.mobilenet_v3_large(weights=weights)
# Congelar capas base
for param in model.parameters():
    param.requires_grad = False
# Reemplazar cabezal
in_features = model.classifier[3].in_features
model.classifier[3] = nn.Linear(in_features, num_classes)
img_size = 224

```

```

return model, img_size

```

3. Función de Entrenamiento

```

def train_model(model, train_loader, val_loader, criterion, optimizer, device, epochs=15, is_i
best_val_loss = float('inf')
patience = 3 # Early Stopping
trigger_times = 0

for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()

        # Inception tiene salidas auxiliares durante el entrenamiento
        if is_inception:
            outputs, aux_outputs = model(inputs)
            loss1 = criterion(outputs, labels)
            loss2 = criterion(aux_outputs, labels)
            loss = loss1 + 0.4 * loss2
        else:
            outputs = model(inputs)
            loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()
        running_loss += loss.item() * inputs.size(0)

    epoch_loss = running_loss / len(train_loader.dataset)

    # Validación
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * inputs.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

```

```

val_loss = val_loss / len(val_loader.dataset)
val_acc = correct / total

print(f"Epoch [{epoch+1}/{epochs}] Train Loss: {epoch_loss:.4f} | Val Loss: {val_loss:.4f}")

# Early Stopping
if val_loss < best_val_loss:
    best_val_loss = val_loss
    trigger_times = 0
    # Guardar mejor modelo temporal
    torch.save(model.state_dict(), 'best_temp.pth')
else:
    trigger_times += 1
    if trigger_times >= patience:
        print("Early stopping! Deteniendo el entrenamiento prematuramente.")
        break

# Cargar los mejores pesos
model.load_state_dict(torch.load('best_temp.pth'))
return model

# 4. Función de Evaluación y Métricas
def evaluate_model(model, test_loader, device, model_name):
    model.eval()
    all_preds = []
    all_labels = []

    # a. y b. Métrica de Accuracy y F1-Score
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds, average='macro')

    # c. Tamaño del modelo
    model_path = f"{model_name}.pth"
    torch.save(model.state_dict(), model_path)
    size_mb = os.path.getsize(model_path) / (1024 * 1024)

    # d. Tiempo de Inferencia
    # Crear un tensor para calentar el modelo y probar
    img_s = 299 if model_name == 'inception_v3' else 224
    dummy_input = torch.randn(1, 3, img_s, img_s).to(device)

    # Calentamiento (necesario en GPUs)
    for _ in range(10):
        _ = model(dummy_input)

    # Medición
    times = []

```



```

with torch.no_grad():
    for _ in range(100):
        start = time.time()
        _ = model(dummy_input)
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        end = time.time()
        times.append((end - start) * 1000) # convertir a milisegundos

avg_time_ms = sum(times) / len(times)

print("-" * 50)
print(f"Resultados Finales para {model_name}:")
print(f"Accuracy de Prueba: {acc:.4f}")
print(f"F1-Score (Macro): {f1:.4f}")
print(f"Tamaño en Disco: {size_mb:.2f} MB")
print(f"Tiempo de Inferencia Promedio: {avg_time_ms:.2f} ms")
print("-" * 50)

return acc, f1, size_mb, avg_time_ms

def main():
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Usando hardware: {device}")

    model_names = ['resnet50', 'inception_v3', 'mobilenet_v3']
    results = {}

    for name in model_names:
        print(f"\n===== Iniciando Evaluación para {name.upper()} =====")
        # 1. Obtener La arquitectura y asignar tamaño de imagen correcto
        model, img_size = get_model(name, num_classes=8)
        model = model.to(device)

        # 2. Cargar datos específicos para este modelo
        train_loader, val_loader, test_loader, num_classes = get_data loaders(img_size=img_size)

        # 3. Configurar entrenamiento (solo pesos no congelados)
        params_to_update = [p for p in model.parameters() if p.requires_grad]
        optimizer = optim.Adam(params_to_update, lr=0.001)
        criterion = nn.CrossEntropyLoss()

        # 4. Ejecutar entrenamiento y validación
        is_inception = (name == 'inception_v3')
        model = train_model(model, train_loader, val_loader, criterion, optimizer, device, epochs)

        # 5. Evaluar en conjunto de prueba
        acc, f1, size_mb, avg_inf_time = evaluate_model(model, test_loader, device, name)
        results[name] = {'acc': acc, 'f1': f1, 'size_mb': size_mb, 'inf_time_ms': avg_inf_time}

if __name__ == '__main__':
    main()

```

Usando hardware: cuda

===== Iniciando Evaluación para RESNET50 =====

Epoch [1/15]	Train Loss: 1.0333	Val Loss: 0.3939	Val Acc: 0.9450
Epoch [2/15]	Train Loss: 0.4474	Val Loss: 0.2037	Val Acc: 0.9683
Epoch [3/15]	Train Loss: 0.3288	Val Loss: 0.1368	Val Acc: 0.9833
Epoch [4/15]	Train Loss: 0.2756	Val Loss: 0.1431	Val Acc: 0.9650
Epoch [5/15]	Train Loss: 0.2486	Val Loss: 0.1123	Val Acc: 0.9750
Epoch [6/15]	Train Loss: 0.2257	Val Loss: 0.0861	Val Acc: 0.9817
Epoch [7/15]	Train Loss: 0.2048	Val Loss: 0.1230	Val Acc: 0.9633
Epoch [8/15]	Train Loss: 0.2031	Val Loss: 0.0924	Val Acc: 0.9783
Epoch [9/15]	Train Loss: 0.1805	Val Loss: 0.0822	Val Acc: 0.9833
Epoch [10/15]	Train Loss: 0.1952	Val Loss: 0.0936	Val Acc: 0.9667
Epoch [11/15]	Train Loss: 0.1654	Val Loss: 0.0672	Val Acc: 0.9833
Epoch [12/15]	Train Loss: 0.1649	Val Loss: 0.0620	Val Acc: 0.9867
Epoch [13/15]	Train Loss: 0.1799	Val Loss: 0.0624	Val Acc: 0.9900
Epoch [14/15]	Train Loss: 0.1613	Val Loss: 0.0566	Val Acc: 0.9900
Epoch [15/15]	Train Loss: 0.1589	Val Loss: 0.0495	Val Acc: 0.9900

Resultados Finales para resnet50:

Accuracy de Prueba: 0.9933

F1-Score (Macro): 0.9934

Tamaño en Disco: 90.04 MB

Tiempo de Inferencia Promedio: 6.25 ms

===== Iniciando Evaluación para INCEPTION_V3 =====

Downloading: "https://download.pytorch.org/models/inception_v3_google-0cc3c7bd.pth" to C:\Users\cvall\.cache\torch\hub\checkpoints\inception_v3_google-0cc3c7bd.pth

100%|██████████| 104M/104M [00:08<00:00, 12.8MB/s]

Epoch [1/15]	Train Loss: 1.7887	Val Loss: 0.9993	Val Acc: 0.8583
Epoch [2/15]	Train Loss: 0.9267	Val Loss: 0.6811	Val Acc: 0.9150
Epoch [3/15]	Train Loss: 0.7147	Val Loss: 0.5020	Val Acc: 0.9500
Epoch [4/15]	Train Loss: 0.6244	Val Loss: 0.4718	Val Acc: 0.9150
Epoch [5/15]	Train Loss: 0.5830	Val Loss: 0.3828	Val Acc: 0.9433
Epoch [6/15]	Train Loss: 0.5342	Val Loss: 0.4037	Val Acc: 0.9283
Epoch [7/15]	Train Loss: 0.5209	Val Loss: 0.4104	Val Acc: 0.9067
Epoch [8/15]	Train Loss: 0.4833	Val Loss: 0.3368	Val Acc: 0.9367
Epoch [9/15]	Train Loss: 0.4580	Val Loss: 0.2847	Val Acc: 0.9567
Epoch [10/15]	Train Loss: 0.4435	Val Loss: 0.2832	Val Acc: 0.9517
Epoch [11/15]	Train Loss: 0.4535	Val Loss: 0.2732	Val Acc: 0.9583
Epoch [12/15]	Train Loss: 0.4388	Val Loss: 0.2801	Val Acc: 0.9550
Epoch [13/15]	Train Loss: 0.4740	Val Loss: 0.2533	Val Acc: 0.9517
Epoch [14/15]	Train Loss: 0.4034	Val Loss: 0.2144	Val Acc: 0.9600
Epoch [15/15]	Train Loss: 0.4223	Val Loss: 0.2635	Val Acc: 0.9433

Resultados Finales para inception_v3:

Accuracy de Prueba: 0.9600

F1-Score (Macro): 0.9594

Tamaño en Disco: 93.28 MB

Tiempo de Inferencia Promedio: 8.01 ms

===== Iniciando Evaluación para MOBILENET_V3 =====

Downloading: "https://download.pytorch.org/models/mobilenet_v3_large-8738ca79.pth" to C:\Users\cvall\.cache\torch\hub\checkpoints\mobilenet_v3_large-8738ca79.pth

```

100%|██████████| 21.1M/21.1M [00:02<00:00, 9.11MB/s]
Epoch [1/15] Train Loss: 0.7677 | Val Loss: 0.2414 | Val Acc: 0.9650
Epoch [2/15] Train Loss: 0.3146 | Val Loss: 0.0875 | Val Acc: 0.9900
Epoch [3/15] Train Loss: 0.2301 | Val Loss: 0.0609 | Val Acc: 0.9833
Epoch [4/15] Train Loss: 0.1970 | Val Loss: 0.0530 | Val Acc: 0.9867
Epoch [5/15] Train Loss: 0.1748 | Val Loss: 0.0428 | Val Acc: 0.9900
Epoch [6/15] Train Loss: 0.1664 | Val Loss: 0.0495 | Val Acc: 0.9883
Epoch [7/15] Train Loss: 0.1536 | Val Loss: 0.0341 | Val Acc: 0.9917
Epoch [8/15] Train Loss: 0.1606 | Val Loss: 0.0355 | Val Acc: 0.9917
Epoch [9/15] Train Loss: 0.1440 | Val Loss: 0.0319 | Val Acc: 0.9917
Epoch [10/15] Train Loss: 0.1359 | Val Loss: 0.0328 | Val Acc: 0.9917
Epoch [11/15] Train Loss: 0.1326 | Val Loss: 0.0345 | Val Acc: 0.9917
Epoch [12/15] Train Loss: 0.1287 | Val Loss: 0.0311 | Val Acc: 0.9933
Epoch [13/15] Train Loss: 0.1342 | Val Loss: 0.0312 | Val Acc: 0.9917
Epoch [14/15] Train Loss: 0.1256 | Val Loss: 0.0281 | Val Acc: 0.9933
Epoch [15/15] Train Loss: 0.1165 | Val Loss: 0.0294 | Val Acc: 0.9917

```

```

-----
Resultados Finales para mobilenet_v3:
Accuracy de Prueba: 0.9933
F1-Score (Macro): 0.9932
Tamaño en Disco: 16.27 MB
Tiempo de Inferencia Promedio: 5.28 ms
-----

```

Task 3

En la industria, el código es solo una herramienta; lo que el cliente paga es su criterio. Basado en los resultados de la Parte 2, redacte un dictamen ejecutivo de 1 a 2 páginas.

Inciso 1

Presente una tabla clara cruzando los 3 modelos versus las 4 métricas evaluadas (Accuracy, F1-Score, Tamaño en MB, Tiempo de Inferencia).

Modelo	Accuracy	F1-Score (Macro)	Tamaño (MB)	Tiempo Inferencia (ms)
ResNet50	99.33%	0.9934	90.04 MB	6.25 ms
InceptionV3	96.00%	0.9594	93.28 MB	8.01 ms
MobileNetV3	99.33%	0.9932	16.27 MB	5.28 ms

Inciso 2

Compare a ResNet e Inception frente a MobileNet. ¿Cuánto "Accuracy" sacrificó usted (si es que sacrificó algo) al usar MobileNet? ¿Cómo se correlaciona el tamaño en Megabytes con la arquitectura matemática que usted describió en la Parte 1?

Al analizar los resultados empíricos, **no se sacrificó exactitud (Accuracy)** en absoluto al usar MobileNetV3 en comparación con ResNet50. De hecho, ambos modelos alcanzaron un excepcional

99.33% de exactitud y un F1-Score virtualmente idéntico (~0.993). Por otro lado, InceptionV3 se quedó rezagado con un 96.00% de exactitud en este conjunto de datos particular.

La correlación entre el tamaño en Megabytes y la arquitectura matemática descrita en la Parte 1 es directa:

- **ResNet50 (90.04 MB) e InceptionV3 (93.28 MB)** dictan su tamaño basados en convoluciones estándar tradicionales ($k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$). Estas operaciones requieren almacenar una inmensa cantidad de parámetros y cruces de canales simultáneos, lo que dispara directamente el espacio en disco que ocupan los pesos.
- **MobileNetV3 (16.27 MB)** es más de **5.5 veces más ligero** que los otros modelos. Esto se explica matemáticamente por el uso estricto de la *Depthwise Separable Convolution*. Al separar unifilarmente el proceso en un filtrado puramente espacial por canal (Depthwise) y una posterior combinación lineal a través de 1×1 (Pointwise), eliminamos la redundancia multiplicativa. Al reducir la cantidad de multiplicaciones de tensores en un factor tremendo, se aminora consecuentemente el número total de parámetros matemáticos almacenados, logrando un modelo sumamente compacto sin perder expresividad.

Inciso 3

Responda al CEO de la startup: Considerando que los agricultores guatemaltecos usarán teléfonos con 2GB de RAM sin internet, ¿qué modelo exacto mandamos a producción y por qué? (Justifique por qué el modelo ganador es viable para Edge AI frente a los perdedores).

Dictamen Ejecutivo Oficial al CEO:

El modelo exacto que debemos empaquetar y mandar a producción para la aplicación final es indiscutiblemente **MobileNetV3**.

Justificación Técnica y de Negocio (Edge AI): Para nuestro cliente meta (agricultores de Guatemala) que utilizarán dispositivos de gama baja (Android con 2GB de RAM) operando en fincas remotas sin conectividad a internet, el factor limitante del éxito del producto no es únicamente su fuerza probabilística, sino su **viabilidad operativa en campo**.

1. **Eficiencia en Memoria RAM y Almacenamiento Libre:** MobileNetV3 pesa únicamente **16.27 MB**, frente a los masivos >90 MB de ResNet o Inception. Cargar en memoria casi 100 MB de redes neuronales estrangulará y colapsará prematuramente los escasos 2GB de RAM de un teléfono Android de entrada (sabiendo que el SO y la UI del celular ya secuestran más de 1GB). MobileNet entra holgadamente en la memoria limpia actual.
2. **Latencia, Temperatura y Batería:** El tiempo de inferencia de 5.28 milisegundos garantiza que, incluso usando procesadores ARM antiguos y sin acelerador neural (NPU), la clasificación será catalogada como "instantánea" en la App. Operar menos FLOPs disminuye diametralmente el estrés térmico del dispositivo expuesto a altas temperaturas de plantación y evita el drenaje precipitado de la batería bajo el sol.

3. **Rendimiento Impecable (Sin Concesiones):** De manera extraordinaria y comercial, el modelo empató con la arquitectura élite ResNet50 y superó al robusto InceptionV3, logrando conservar un asombroso 99.33% de exactitud en datos de validación cruzada.

En conclusión, MobileNetV3 es el gran ganador. Entregará inferencia y confiabilidad dignas de un clúster servidor de IA, pero empaquetado matemáticamente de tal manera que sobrevive y opera 100% *Offline* nativamente en la palma de las manos de los agricultores.